A Cache in the Weights? Dynamic Evaluation and Temporal Locality in Small Language Models

Ruben Tous
Universitat Politècnica de Catalunya, Barcelona, Spain

Draft, November 10, 2026

Abstract

Dynamic evaluation, or test-time training, lets a language model keep learning from the text it reads at inference time. The cost is forgetting, as each update can overwrite earlier knowledge, and this is usually treated as a problem. However, the inputs a model receives at inference time often have temporal locality. A book, a long conversation or a code base stays on the same characters, names and vocabulary far longer than an attention window lasts. In a small model, with little room in its weights, locality and forgetting together could be exploited to obtain the behaviour of a cache memory. The weights would hold the recent context beyond the window, and new material would push out old material. We ask whether this cache behaviour arises on its own in a standard transformer that takes plain gradient steps as it reads. We test this with small Pythia models reading PG-19 books. We split the books into short chunks and score each one without the preceding text in context, so any help from earlier chunks must come through the weights. We then compare reading the chunks in order with reading them shuffled, which removes their locality. We find support for the cache hypothesis. The weights hold a memory of the current book beyond the attention window, stored mostly in the MLPs. This memory depends on the order of the input and fades as other books are read, halving within a few thousand tokens. As expected, it competes with general knowledge for the same weights. Knowledge of the same domain is largely preserved, while out-of-domain text is partly forgotten, and a stronger cache is hard to get without more of this forgetting. Decoupling the two could give a stronger cache and a better use of temporal locality, reducing the model size required for a given performance.

1 Introduction

Dynamic evaluation [12, 13, 18] and test-time training [6, 2, 22] let a language model keep learning from the text it reads at inference time, and both report lower loss. The cost is forgetting: each update can overwrite earlier knowledge, the catastrophic interference of sequential training [15], and this is usually treated as a problem.

However, what a model reads at inference time often has temporal locality: a book keeps its characters, places and turns of phrase for hundreds of thousands of tokens, and so do a long conversation or a code base, far longer than an attention window lasts. Computer memory hierarchies exploit this property with a small cache that holds recently used data and lets older data be pushed out [5]. A transformer has such a cache in its attention window, but the window is short and everything before it is lost. In a small model, with little room in its weights, locality and forgetting together could give the behaviour of a cache memory: the weights would hold the recent context beyond the window, and new material would push out old material.

We ask whether this cache behaviour arises on its own in a standard transformer that takes plain gradient steps as it reads. If it does, three things should be true. The benefit should depend on *locality*: it should be larger when the input stays on the same material than when the same text arrives in a random order. New material should push out old material, which we call *eviction*; this is the useful side of forgetting. And the cache should leave the knowledge the model already had largely intact; loss of that general knowledge is what we call *forgetting*.

We test this with small Pythia models [4] reading PG-19 books [17]. Each 256-token chunk is scored on its own, without the previous chunks in the context, so anything that helps from further back must come through the weights; after scoring a chunk, the model takes one update step on it. We compare reading the chunks in order with reading them shuffled, which removes their locality. We ask whether dynamic evaluation helps more when the input has locality than when it is shuffled, whether it evicts old content like a cache, what it costs

in general knowledge, where in the network the memory lives and whether a knob sets its size, and whether it matters more in small models or can replace a larger one.

Contributions, one per group of findings:

- a local memory beyond the window, tied to the current book and more than a token-count cache (Section 4.1);
- its location: the MLPs, where updating only them gives more cache and more forgetting (Section 4.2);
- the gain as a function of the locality of the input (Section 4.3);
- eviction and forgetting measured with frozen snapshots of the learner (Sections 4.4 and 4.5);
- update rules across sizes, and a trade-off between cache and forgetting (Sections 4.6 and 4.7);
- long books: what dynamic evaluation is worth against a larger model (Section 4.8).

2 Related work

Dynamic evaluation. Krause et al. [12, 13] adapt a language model to the sequence it predicts with gradient steps at test time; Rannen-Triki et al. [18] revisit this for larger models and describe it as memory in the weights. We use the same protocol (score, then update) and ask what the updated weights remember, for how long, where, and at what cost.

Cache language models. Recent words [14] or recent hidden states [9] can be mixed into a model's prediction; our unigram cache is the simplest such baseline.

Test-time training. Test-time training adapts a model to each test input with a self-supervised loss [21]. For language models, In-Place TTT [6] updates the final projection of the MLP blocks, qTTT [2] the query projections, and TTT-E2E [22] meta-learns a model for test-time training. That the MLPs hold most of the memory agrees with In-Place TTT's choice and with feed-forward layers as key-value memories [8].

Fast weights. Fast weights hold short-term memory on top of slow weights [10, 20, 1, 19]; we do not test a separate fast part here (Section 7).

3 Setup

Models. Pythia-14M, 31M and 70M [4] (14.07M, 30.49M and 70.43M parameters, 6 layers each, same tokenizer), in float32 on CPU, seed 0.

Data. All books are from PG-19 [17]. The *main set* is 25 test books, each cut into 78 chunks of 256 tokens (its first 19,968 tokens). The *long set* is 5 test books of 100k tokens (390 chunks each), and the *very long set* is 4 test books of 200k tokens (781 chunks each); all 5 long books and 2 of the 4 very long books are also in the main set, cut there to their first 78 chunks. Learning rates are picked on *val-pick*, 10 PG-19 validation books of 78 chunks each. Two held-out sets are never trained on: 76 chunks of WikiText-103's test split [16], detokenized, and 6 other validation books of 15 chunks each. Pythia was trained on the Pile [7], whose PG-19 component has 28,602 documents [3], the size of PG-19's training split; the test books are therefore very likely not in Pythia's training data, although the authors do not say so and other copies of the books were not checked.

Protocol. Each chunk is scored on its own, without earlier chunks in the context. With dynamic evaluation, after scoring a chunk the model takes one update step on the chunk's mean loss, so every token is predicted before it is trained on. We call the updated model the *learner*; the frozen model reads the same chunks without updates.

Update rules and parameter subsets. *Plain SGD* uses no momentum or weight decay. *Adam* [11] updates all parameters with a fresh state at the start of each stream. *Normalized SGD* moves every tensor by the same relative amount per step, $W \leftarrow W - \eta \, \|W\| \, g/\|g\|$. With plain SGD, some runs update only part of Pythia-14M: the output embedding, everything but the output embedding, the MLPs of all layers, the attention of all layers, or one layer's MLP.

Stream orders. The main set's chunks are read in four orders: book after book ($K = 1$), 5 books interleaved chunk by chunk ($K = 5$), all 25 interleaved ($K = 25$), and one random order (shuffled). The long and very long sets are read book after book.

Snapshot probes. To see what the weights hold at a given point, we copy them and score other text with the copy frozen, while the learner continues. Snapshots are taken every 40 chunks and scored on the held-out sets. For eviction (Section 4.4), each of 20 main books A is split into a first half A1 (40 chunks) and a second half A2 (38 chunks); the learner reads A1, then I tokens of PG-19 validation books, and a snapshot scores A2 and the same span C2 of a control book C that it never read.

Learning rates. Each learning rate is the value with the lowest mean loss under dynamic evaluation on val-pick, from a grid of 1 and 3 times powers of ten. The grid is extended until both neighbours of the best value have been tried, so every pick lies inside its grid. Each parameter subset, update rule and size has its own pick.

Metrics. The *gain* measures how much dynamic evaluation helps: the frozen loss minus the loss under dynamic evaluation on the same tokens, in nats/token, against each model's own frozen loss. *Distance bins* sort tokens by how far back the same token id last occurred: in the same chunk (visible to attention), in an earlier chunk less than 1k, 1k to 10k, 10k to 100k or at least 100k tokens back, or never; the 1000 most frequent ids are left out of the earlier-chunk bins. Earlier-chunk bins are split by whether that occurrence was in the same book or an earlier one. *Locality* measures how much of the gain needs the order of the text: the gain in reading order minus the gain on the same chunks shuffled. The *memory of A* measures what the snapshot still knows about book A beyond general adaptation: its gain on A2 minus its gain on C2. *Forgetting* measures lost general knowledge: a snapshot's loss on a held-out set minus the pretrained model's, averaged over the snapshots in the second half of the stream (positive means knowledge lost). The *exchange rate* measures what a run gains per unit of knowledge it loses: its gain on the stream divided by its WikiText forgetting.

Error bars. Means are over tokens. Standard errors (SE) are over books, from per-book means or, when two runs read the same chunks, from paired per-book differences; books with fewer than 50 tokens in a bin are left out. For forgetting, SE are over the WikiText chunks or the 6 held-out books. A † marks a value under 2 SE, which we do not treat as a difference.

4 Results

4.1 A memory beyond the window, and it is local

If the updated weights hold a memory of the text, tokens seen recently in an earlier chunk should be predicted better, and the benefit should depend on the order of the chunks. We bin tokens by the distance to their previous occurrence and compare reading in order with reading the same chunks shuffled.

With plain SGD at lr 10^{-4}, Pythia-14M gains $+0.100 \pm 0.006$ nats/token over its frozen loss of 4.393 (Table 1). The gain falls with distance: $+0.307 \pm 0.011$ for rare tokens last seen less than 1k tokens back, $+0.144 \pm 0.008$ between 1k and 10k, and $+0.009 \pm 0.006^{\dagger}$ between 10k and 100k. Tokens never seen before lose -0.059 ± 0.013, and tokens repeated within their own chunk lose -0.111 ± 0.014. Within a book the gain grows from $+0.013 \pm 0.008^{\dagger}$ in its first 1k tokens to $+0.139 \pm 0.008$ in its last 1k, while the shuffled learner's gain stays between $+0.055$ and $+0.071$ (Figure 1b).

Shuffling keeps 62% of the overall gain, but only 20% less than 1k tokens back and 24% between 1k and 10k (locality $+0.245 \pm 0.009$ and $+0.109 \pm 0.006$). Most of the overall gain is therefore adaptation to PG-19 text in general, and most of the long-range gain needs the order.

bin (previous occurrence)	tokens	frozen loss	in order	shuffled	locality	unigram cache
all tokens	497,250	4.393	$+0.100_{\pm.006}$	$+0.062_{\pm.003}$	$+0.038_{\pm.004}$	$+0.018_{\pm.004}$
same chunk	9,886	5.006	$-0.111_{\pm.014}$	$-0.133_{\pm.010}$	$+0.022^{\dagger}_{\pm.013}$	$+0.096_{\pm.017}$
earlier, $<$ 1k	17,036	7.335	$+0.307_{\pm.011}$	$+0.062_{\pm.007}$	$+0.245_{\pm.009}$	$+0.461_{\pm.024}$
earlier, 1k-10k	33,973	6.997	$+0.144_{\pm.008}$	$+0.035_{\pm.005}$	$+0.109_{\pm.006}$	$+0.059_{\pm.007}$
earlier, 10k-100k	44,109	7.113	$+0.009^{\dagger}_{\pm.006}$	$+0.026_{\pm.005}$	$-0.017_{\pm.006}$	$-0.051_{\pm.000}$
earlier, $\geq$ 100k	13,125	7.824	$-0.039_{\pm.009}$	$-0.017_{\pm.008}$	$-0.023_{\pm.008}$	$-0.051_{\pm.000}$
never seen	20,976	8.190	$-0.059_{\pm.013}$	$-0.055_{\pm.009}$	$-0.004^{\dagger}_{\pm.010}$	$-0.051_{\pm.000}$

Table 1: Gain over frozen Pythia-14M by distance to the token's previous occurrence (nats/token, ± SE over 25 books). Dynamic evaluation: plain SGD on all parameters, lr 10^{-4}, in reading order ($K = 1$) and with the chunks shuffled; locality = in order − shuffled. Unigram cache: frozen model mixed with a decaying token-count cache; where the cache has no count for a token, mixing it in costs exactly $-\log 0.95 = 0.051$ nats, so those bins have a constant loss and an SE of 0. †: under 2 SE.

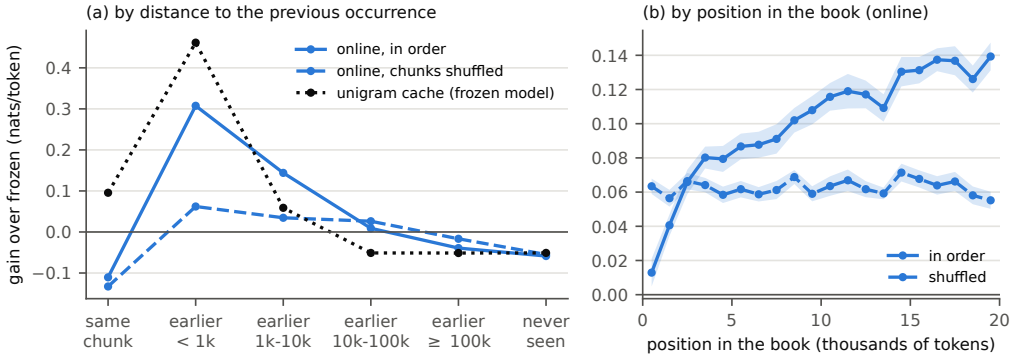


Figure 1: Gain over frozen Pythia-14M with plain SGD (lr 10^{-4}). (a) By distance to the token's previous occurrence, for dynamic evaluation in order and shuffled and for the unigram cache. (b) By position in the book, mean over the 24 books that follow a book switch, ±1 SE over books.

The current book. A token may be predicted better because it was seen recently, or because it belongs to the book being read. Splitting the earlier-chunk bins by book separates the two (Table 2). Between 1k and 10k tokens back, the locality is $+0.120 \pm 0.007$ for tokens last seen in the same book and $+0.035 \pm 0.009$ for tokens last seen in an earlier book. Between 10k and 100k tokens back, same-book tokens keep $+0.036 \pm 0.010$ of locality and earlier-book tokens have none (-0.024 ± 0.006). The memory is tied mostly to the current book, although distance still matters within it.

More than a count cache. A cache of recent token counts mixed into the frozen model ($p = 0.95\, p_{\text{model}} + 0.05\, p_{\text{cache}}$, counts halving every 1k tokens) is the simplest way to use the recent past. It is the better copier less than 1k tokens back ($+0.461 \pm 0.024$ against $+0.307$), but from 1k tokens on dynamic evaluation gains more ($+0.144$ against $+0.059 \pm 0.007$ between 1k and 10k; paired difference $+0.085 \pm 0.008$), and $+0.082 \pm 0.004$ more overall (Table 1).

4.2 Where the memory lives

If the memory sits in particular weights, updating only those should keep it. We update one part of Pythia-14M at a time, each part at its own learning rate (Table 3).

Freezing the output embedding costs little (-0.006 ± 0.001 overall against all parameters). The output embedding alone is a short-range cache: $+0.252 \pm 0.017$ less than 1k tokens back but $+0.072 \pm 0.006$ between 1k and 10k (same book), with the largest loss on never-seen tokens (-0.143 ± 0.012). Attention alone keeps less of the long-range gain ($+0.110 \pm 0.009$ between 1k and 10k, same book). Updating only the MLPs (lr 3×10^{-4}) gains more than updating all parameters (lr 10^{-4}): $+0.415 \pm 0.016$ against $+0.307$ less than 1k tokens back (paired difference $+0.108 \pm 0.008$) and $+0.107 \pm 0.007$ against $+0.100$ overall ($+0.007 \pm 0.002$).

One layer's MLP alone keeps between 36% and 93% of the full learner's gain less than 1k tokens back,

bin	previous occurrence	tokens	in order	shuffled	locality	unigram cache
earlier, $<$ 1k	same book	16,967	$+0.308_{\pm.011}$	$+0.062_{\pm.007}$	$+0.246_{\pm.009}$	$+0.462_{\pm.024}$
earlier, 1k-10k	same book	29,577	$+0.154_{\pm.008}$	$+0.034_{\pm.005}$	$+0.120_{\pm.007}$	$+0.070_{\pm.009}$
earlier, 1k-10k	earlier book	4,396	$+0.076_{\pm.012}$	$+0.040_{\pm.009}$	$+0.035_{\pm.009}$	$-0.016_{\pm.004}$
earlier, 10k-100k	same book	4,798	$+0.060_{\pm.011}$	$+0.024_{\pm.007}$	$+0.036_{\pm.010}$	$-0.050_{\pm.000}$
earlier, 10k-100k	earlier book	39,311	$+0.003^{\dagger}_{\pm.006}$	$+0.026_{\pm.005}$	$-0.024_{\pm.006}$	$-0.051_{\pm.000}$

Table 2: Gain over frozen Pythia-14M by whether the token's previous occurrence is in the same book or an earlier one (± SE over books); same runs as Table 1. The earlier-book part of the $<$ 1k bin (69 tokens) is omitted. †: under 2 SE.

parameters updated	parameters	lr	all tokens	earlier, $<$ 1k	1k-10k, same book	never seen
all parameters	14,067,712	10^{-4}	$+0.100_{\pm.006}$	$+0.307_{\pm.011}$	$+0.154_{\pm.008}$	$-0.059_{\pm.013}$
output embedding only	6,438,912	3×10^{-3}	$+0.072_{\pm.005}$	$+0.252_{\pm.017}$	$+0.072_{\pm.006}$	$-0.143_{\pm.012}$
all but output embedding	7,628,800	10^{-4}	$+0.094_{\pm.005}$	$+0.294_{\pm.011}$	$+0.151_{\pm.009}$	$-0.058_{\pm.013}$
MLPs only	790,272	3×10^{-4}	$+0.107_{\pm.007}$	$+0.415_{\pm.016}$	$+0.175_{\pm.011}$	$-0.077_{\pm.014}$
attention only	396,288	3×10^{-4}	$+0.066_{\pm.005}$	$+0.259_{\pm.011}$	$+0.110_{\pm.009}$	$-0.074_{\pm.014}$
MLP of layer 1 only	131,712	10^{-3}	$+0.053_{\pm.004}$	$+0.236_{\pm.012}$	$+0.113_{\pm.007}$	$-0.055_{\pm.010}$
MLP of layer 2 only	131,712	10^{-3}	$+0.069_{\pm.004}$	$+0.242_{\pm.011}$	$+0.131_{\pm.008}$	$-0.051_{\pm.010}$
MLP of layer 3 only	131,712	3×10^{-4}	$+0.055_{\pm.004}$	$+0.204_{\pm.009}$	$+0.093_{\pm.007}$	$-0.056_{\pm.011}$
MLP of layer 4 only	131,712	10^{-3}	$+0.063_{\pm.004}$	$+0.187_{\pm.010}$	$+0.101_{\pm.007}$	$-0.063_{\pm.010}$
MLP of layer 5 only	131,712	3×10^{-3}	$+0.073_{\pm.006}$	$+0.285_{\pm.014}$	$+0.121_{\pm.009}$	$-0.090_{\pm.014}$
MLP of layer 6 only	131,712	10^{-3}	$+0.048_{\pm.004}$	$+0.111_{\pm.008}$	$+0.053_{\pm.006}$	$-0.084_{\pm.012}$

Table 3: Gain over frozen Pythia-14M when only some parameters are updated with plain SGD, in reading order ($K = 1$), each subset at its own val-pick learning rate (± SE over books). Columns are distance bins; "1k-10k, same book" counts tokens whose previous occurrence is in the same book. †: under 2 SE.

and between 35% and 85% between 1k and 10k (same book); every single layer is below the full learner there (Figure 2a). The strongest is layer 5 ($+0.285 \pm 0.014$, paired difference with all parameters -0.023 ± 0.009), and the last layer is the weakest ($+0.111 \pm 0.008$).

MLPs only against all parameters. A larger gain could come from a stronger cache or from better general adaptation, and a stronger cache might cost more general knowledge. We read the shuffled stream with both learners and probe both with snapshots (Table 4). Updating only the MLPs gives more locality: $+0.334 \pm 0.011$ against $+0.245 \pm 0.009$ less than 1k tokens back (paired difference $+0.088 \pm 0.005$) and $+0.145 \pm 0.009$ against $+0.120 \pm 0.007$ between 1k and 10k, same book ($+0.025 \pm 0.003$). Shuffled, it gains slightly less overall (-0.003 ± 0.001). It also forgets more: WikiText forgetting is $+0.235 \pm 0.007$ against $+0.196 \pm 0.006$ (paired $+0.039 \pm 0.003$), and the held-out books lose $+0.022 \pm 0.002$ more (paired), although neither run's own value on them is beyond 2 SE. Its exchange rate is lower: 0.45 against 0.51 in order, 0.41 against 0.55 shuffled.

The two learners use different learning rates, each its own val-pick (val-pick loss 4.2506 for the MLPs at 3×10^{-4} and 4.2565 for all parameters at 10^{-4}). All parameters at 3×10^{-4} is already worse on val-pick (4.2732). A possible reason, not tested: with plain SGD at 14M, attention takes 43% of the squared update norm and the MLPs 50% (Section 4.6), which may limit the step the full learner can use.

4.3 Locality as a variable

A cache should help more the longer the input stays on the same material. We read the same chunks with more and more books interleaved, and the long books in order (Figure 2b).

With plain SGD on all parameters at lr 10^{-4}, the gain falls at every step: $+0.176 \pm 0.023$ on the long books, $+0.100 \pm 0.006$ with $K = 1$, $+0.074 \pm 0.004$ with $K = 5$, $+0.065 \pm 0.003$ with $K = 25$ and $+0.062 \pm 0.003$ shuffled. On the main set the paired drops are -0.026 ± 0.002 from $K = 1$ to $K = 5$, -0.009 ± 0.002 from $K = 5$ to $K = 25$ and -0.003 ± 0.001 from $K = 25$ to shuffled; interleaving only 5 books removes most of the in-order advantage. The long books are other books than the main set, so the step from them to $K = 1$ is not paired.

4.4 Eviction

If dynamic evaluation acts as a cache, reading other text should push a book out of memory. We let the model read the first half of a book, then other books, and check with a frozen snapshot how well it still predicts the

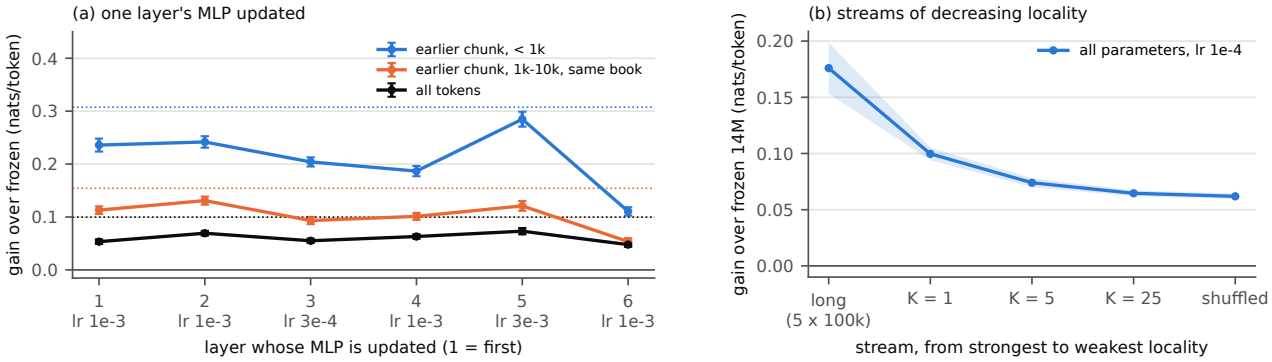


Figure 2: Gain over frozen Pythia-14M with plain SGD, ±1 SE over books. (a) Only one layer's MLP updated, each at its own learning rate (below the layer number); dotted lines: all parameters at lr 10^{-4}. (b) All parameters at lr 10^{-4}, on streams of decreasing locality: 5 long books of 100k tokens, then the main set with books in order ($K = 1$), 5 or 25 books interleaved, and shuffled.

quantity	bin / stream	MLPs only, lr 3×10^{-4}	all, lr 10^{-4}	MLPs only − all
gain shuffled	all tokens	$+0.059_{\pm.003}$	$+0.062_{\pm.003}$	$-0.003_{\pm.001}$
	earlier, < 1k	$+0.081_{\pm.011}$	$+0.062_{\pm.007}$	$+0.019_{\pm.005}$
	1k-10k, same book	$+0.030_{\pm.006}$	$+0.034_{\pm.005}$	$-0.004^{\dagger}_{\pm.002}$
	never seen	$-0.071_{\pm.010}$	$-0.055_{\pm.009}$	$-0.016_{\pm.003}$
locality	all tokens	$+0.048_{\pm.004}$	$+0.038_{\pm.004}$	$+0.010_{\pm.001}$
	earlier, < 1k	$+0.334_{\pm.011}$	$+0.245_{\pm.009}$	$+0.088_{\pm.005}$
	1k-10k, same book	$+0.145_{\pm.009}$	$+0.120_{\pm.007}$	$+0.025_{\pm.003}$
	never seen	$-0.006^{\dagger}_{\pm.011}$	$-0.004^{\dagger}_{\pm.010}$	$-0.002^{\dagger}_{\pm.004}$
WikiText forgetting	$K = 1$	$+0.235_{\pm.007}$	$+0.196_{\pm.006}$	$+0.039_{\pm.003}$
	shuffled	$+0.144_{\pm.005}$	$+0.112_{\pm.004}$	$+0.031_{\pm.002}$
held-out books forgetting	$K = 1$	$+0.016^{\dagger}_{\pm.009}$	$-0.006^{\dagger}_{\pm.010}$	$+0.022_{\pm.002}$
	shuffled	$-0.034_{\pm.007}$	$-0.045_{\pm.008}$	$+0.011_{\pm.001}$
exchange rate	$K = 1$	0.45	0.51	−0.06
	shuffled	0.41	0.55	−0.14

Table 4: Pythia-14M with plain SGD updating only the MLPs or all parameters, each at its own val-pick learning rate. Gain shuffled and locality (in order − shuffled) by distance bin, ± SE over books; forgetting (snapshot − pretrained loss, mean over the second half of the stream) on detokenized WikiText and the 6 held-out books, ± SE over held-out chunks or books; exchange rate = stream gain / WikiText forgetting. Last column: paired difference. The in-order gains are in Table 3. †: under 2 SE.

book's second half, compared with a book it never read (Figure 3).

With plain SGD on all parameters at lr 10^{-4}, the memory of A is $+0.097\pm0.013$ right after A1. It halves after about 6.1k tokens of other books and then declines slowly: $+0.044\pm0.007$ after 10k tokens, $+0.031\pm0.006$ after 100k and $+0.016\pm0.008$ after 300k. The snapshots never gain on the control book (-0.030 ± 0.007 right after A1, between -0.032 and -0.011 later), so the decline is the loss of A rather than general adaptation catching up. The memory is mostly about tokens that occurred in A1: rare tokens of A2 whose id occurs in A1 carry $+0.223\pm0.029$ after A1, and other rare tokens $+0.056\pm0.011$; the former halve after about 2.9k tokens. A power law in I fits the memory better than an exponential with a floor (χ^2 2.0 against 8.7, with 4 degrees of freedom each), but the 7 points share the same 20 books, so this only ranks the two shapes.

4.5 Forgetting

A cache should leave the rest of memory intact. We score frozen snapshots of the learner on text it never reads: WikiText, outside the domain, and unseen PG-19 books, inside it (Table 5).

With plain SGD at lr 10^{-4}, Pythia-14M reading the main set in order forgets $+0.196\pm0.006$ nats/token on WikiText (pretrained loss 4.633), about 2.0 times its gain on the stream (exchange rate 0.51). On the held-out PG-19 books it loses nothing ($-0.006\pm0.010^{\dagger}$). The shuffled learner forgets less on WikiText ($+0.112\pm0.004$) and improves on the unseen books (-0.045 ± 0.008): the part of the gain that survives shuffling is adaptation

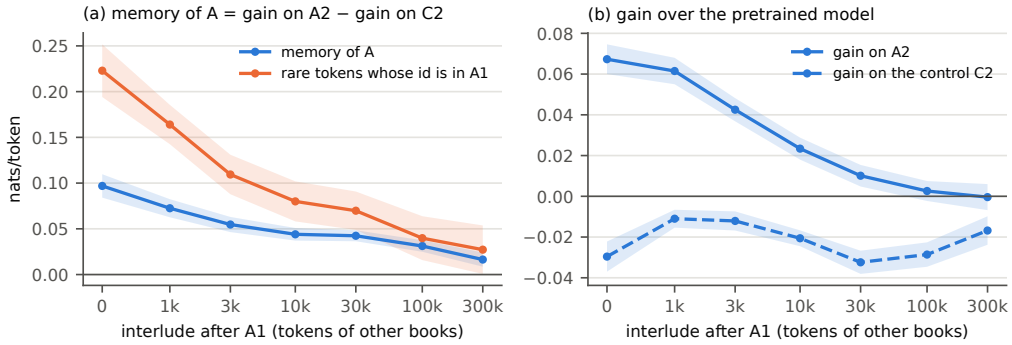


Figure 3: Eviction probed with frozen snapshots (Pythia-14M, plain SGD, all parameters, lr 10^{-4}; mean of 20 books, bands ±1 SE over books), against the number of tokens of other books read after A1. (a) Memory of A (gain on A2 − gain on the never-read control C2, same snapshot), and the same for rare tokens whose id occurs in A1. (b) Gain over the pretrained model on A2 and on C2.

stream	stream gain	WikiText forgetting	held-out books forgetting	exchange rate
$K = 1$	$+0.100_{\pm.006}$	$+0.196_{\pm.006}$	$-0.006^{\dagger}_{\pm.010}$	0.51
shuffled	$+0.062_{\pm.003}$	$+0.112_{\pm.004}$	$-0.045_{\pm.008}$	0.55
very long (4 × 200k)	$+0.170_{\pm.011}$	$+0.268_{\pm.008}$	$+0.052_{\pm.013}$	0.63

Table 5: Forgetting of held-out text by Pythia-14M with plain SGD on all parameters (lr 10^{-4}): snapshot loss − pretrained loss, mean over the snapshots in the second half of the stream, ± SE over the WikiText chunks or the 6 held-out books; positive means knowledge lost. Stream gain: over the frozen loss on the stream, ± SE over books. Exchange rate = stream gain / WikiText forgetting. †: under 2 SE.

to PG-19 text, and it carries over to unseen books. Reading in order forgets more than shuffled on both sets (shuffled − in order: -0.084 ± 0.003 on WikiText, -0.040 ± 0.002 on the books), at about the same exchange rate (0.51 and 0.55). On the very long set (800k tokens), WikiText forgetting averages +0.143 over the snapshots taken in the first book and +0.273, +0.269 and +0.267 in the next three: it levels off after the first book. There the held-out books also lose a little ($+0.052 \pm 0.013$).

4.6 Update rules and model size

With plain SGD, a learning rate small enough for some weights may be too small for others, and the balance changes with model size. We log the size of each update and compare three update rules at three sizes, each with its own learning rate (Table 6, Figure 4).

Over the first 40 chunks, plain SGD at each size's pick moves the MLP weights by 1.2×10^{-4} of their norm per step at 14M, 9.9×10^{-5} at 31M and 1.1×10^{-6} at 70M, about 110 times less at 70M than at 14M; at 70M attention takes 78% of the squared update norm and the MLPs 4%. With plain SGD, Pythia-70M gains $+0.025 \pm 0.002$ and has almost no locality ($+0.016 \pm 0.003$ less than 1k tokens back). Normalized SGD more than doubles 70M's gain ($+0.057 \pm 0.004$; paired difference $+0.033 \pm 0.002$) and gives it locality ($+0.084 \pm 0.006$), still far less than at 14M and 31M under the same rule ($+0.372 \pm 0.011$ and $+0.418 \pm 0.013$), which are the largest of any rule. Adam has no cache at any size: its locality less than 1k tokens back is -0.052 ± 0.009 at 14M, -0.070 ± 0.011 at 31M and -0.140 ± 0.010 at 70M.

From 14M to 31M, plain SGD's locality does not change beyond noise (31M − 14M: $+0.017 \pm 0.014^{\dagger}$ less than 1k tokens back, $+0.002 \pm 0.006^{\dagger}$ overall; SE unpaired across models); it falls at 70M under every rule. The smaller model does not pay more general knowledge per nat of gain: the exchange rate at 14M is 0.51, 0.61 and 0.36 (plain SGD, Adam, normalized SGD), against 0.37, 0.51 and 0.26 at 31M. At 70M it is 2.57, 0.47 and 1.48, with much smaller gains.

4.7 A trade-off between cache and general knowledge

If the cache and general knowledge share the same weights, filling one should erode the other. We place every run with snapshots on two axes: how much cache it has (locality less than 1k tokens back) and how much WikiText

size	rule	lr	gain, $K = 1$	locality, < 1k	locality, all	WikiText forgetting	exch. rate, $K = 1$	shuffled
14M	plain SGD	10^{-4}	$+0.100_{\pm.006}$	$+0.245_{\pm.009}$	$+0.038_{\pm.004}$	$+0.196_{\pm.006}$	0.51	0.55
	Adam	3×10^{-6}	$+0.086_{\pm.005}$	$-0.052_{\pm.009}$	$-0.004^{\dagger}_{\pm.004}$	$+0.141_{\pm.005}$	0.61	0.91
	normalized SGD	10^{-4}	$+0.113_{\pm.008}$	$+0.372_{\pm.011}$	$+0.056_{\pm.006}$	$+0.316_{\pm.008}$	0.36	0.29
31M	plain SGD	3×10^{-4}	$+0.085_{\pm.005}$	$+0.262_{\pm.010}$	$+0.040_{\pm.004}$	$+0.228_{\pm.005}$	0.37	0.31
	Adam	3×10^{-6}	$+0.070_{\pm.004}$	$-0.070_{\pm.011}$	$-0.007_{\pm.003}$	$+0.137_{\pm.004}$	0.51	0.75
	normalized SGD	10^{-4}	$+0.092_{\pm.007}$	$+0.418_{\pm.013}$	$+0.060_{\pm.005}$	$+0.348_{\pm.006}$	0.26	0.13
70M	plain SGD	10^{-5}	$+0.025_{\pm.002}$	$+0.016_{\pm.003}$	$+0.004_{\pm.002}$	$+0.010_{\pm.002}$	2.57	2.67
	Adam	3×10^{-7}	$+0.029_{\pm.002}$	$-0.140_{\pm.010}$	$-0.015_{\pm.002}$	$+0.060_{\pm.003}$	0.47	0.68
	normalized SGD	3×10^{-6}	$+0.057_{\pm.004}$	$+0.084_{\pm.006}$	$+0.018_{\pm.003}$	$+0.039_{\pm.002}$	1.48	1.52

Table 6: Three update rules at three sizes, all parameters, each at its val-pick learning rate: gain with $K = 1$ over each model's frozen loss and locality (in order − shuffled, paired), ± SE over books; WikiText forgetting ($K = 1$, second half of the stream, ± SE over chunks); exchange rate (stream gain / WikiText forgetting) in order and shuffled. †: under 2 SE.

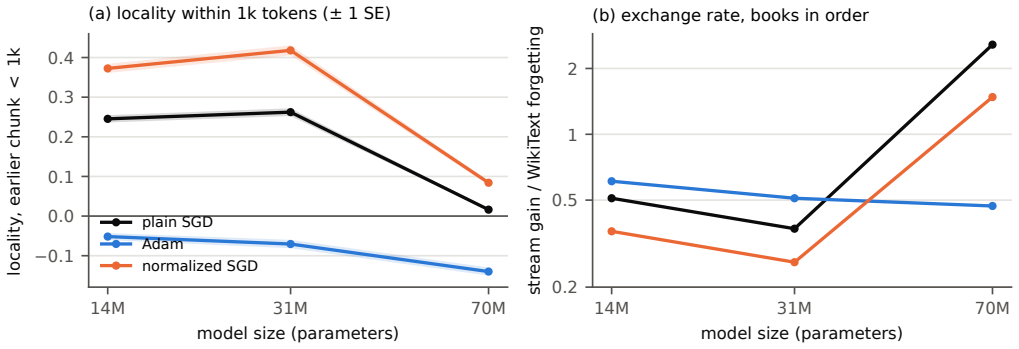


Figure 4: (a) Locality less than 1k tokens back against model size for three update rules, each at its own learning rate, ±1 SE over books. (b) Exchange rate (stream gain / WikiText forgetting, $K = 1$; log scale).

it forgets (Figure 5).

Within 14M the order is the same on both axes: Adam (−0.052, +0.141), plain SGD on all parameters (+0.245, +0.196), plain SGD on the MLPs only (+0.334, +0.235) and normalized SGD (+0.372, +0.316). Within 31M the same holds for Adam, plain SGD and normalized SGD. At 70M plain SGD and normalized SGD follow it, but Adam forgets more than both ($+0.060 \pm 0.003$) with negative locality. Across sizes the points do not lie on one curve: at 70M every rule forgets far less for a given locality. With ten points, one seed and one model family, this is a consistent ordering within the 14M and 31M settings we tried, not a law, and it does not hold for Adam at 70M.

4.8 Long books and model size

A cache is worth more the longer the material stays the same, and a small model might use it to approach a larger one. We let Pythia-14M and Pythia-31M read books of 200k tokens with dynamic evaluation and compare them with frozen 14M, 31M and 70M (Table 7, Figure 6).

With plain SGD at lr 10^{-4}, Pythia-14M gains $+0.170 \pm 0.011$ over its frozen loss and closes 49% of the gap to a frozen Pythia-31M. The share grows from 7% in the first 10k tokens of a book to 79% between 160k and 170k tokens, and is 56% in the last 10k; frozen 31M stays ahead overall ($+0.180 \pm 0.019$). Pythia-31M with dynamic evaluation (plain SGD, lr 3×10^{-4}) gains about as much ($+0.168 \pm 0.013$ over its frozen loss), stays ahead of 14M with dynamic evaluation ($+0.348 \pm 0.013$), and does not reach a frozen Pythia-70M ($+0.054 \pm 0.010$ behind).

5 Discussion

Does dynamic evaluation help more with locality than with random input? Yes: the gain from more than 1k tokens back mostly disappears when the chunks are shuffled, and the overall gain falls at every step from long books to shuffled chunks (Sections 4.1 and 4.3).

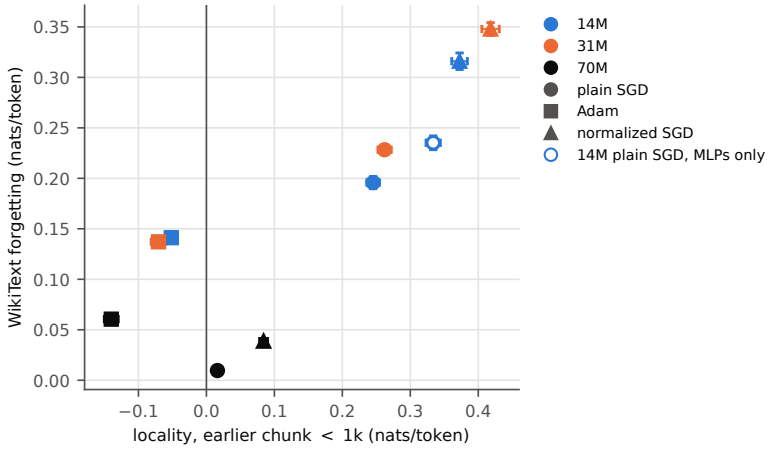


Figure 5: Locality less than 1k tokens back against WikiText forgetting ($K = 1$, second half of the stream) for every run with snapshots: three update rules (marker) at three sizes (colour) on all parameters, and plain SGD on the MLPs only at 14M (open circle); each at its own learning rate. Bars: ±1 SE.

run	loss	gain over frozen 14M	share of 14M-31M gap
frozen 14M	4.419		
14M dyn. eval., lr 10^{-4}	4.249	$+0.170_{\pm.011}$	49%
frozen 31M	4.069	$+0.351_{\pm.010}$	100%
31M dyn. eval., lr 3×10^{-4}	3.901	$+0.518_{\pm.013}$	148%
frozen 70M	3.846	$+0.573_{\pm.013}$	163%

Table 7: Four PG-19 books of 200k tokens, read book after book. Loss, gain over frozen Pythia-14M (± SE over 4 books, paired) and share of the gap between frozen 14M and frozen 31M closed. Dynamic-evaluation runs: plain SGD on all parameters.

Does it evict like a cache? Yes for the memory of a specific book: it halves after a few thousand tokens of other books, with a long tail (Section 4.4). Unlike a cache, filling it overwrites general knowledge outside the domain: WikiText forgetting is about twice the gain on the stream, and within 14M and 31M a stronger cache forgets more (Sections 4.5 and 4.7).

Is the effect larger in small models? Not between 14M and 31M: locality does not change beyond noise, and 14M does not pay more forgetting per nat of gain. 70M has less cache, but how much depends on the update rule and the step size it allows (Section 4.6).

Is there a knob? Partly: the parameter subset and the update rule move the size of the cache, but within 14M and 31M every setting we tried moves cache and forgetting together (Section 4.7). No knob tested here separates the cache from general knowledge.

Can a small model with dynamic evaluation replace a larger one? Not at these sizes: on long books dynamic evaluation is worth about half the step from 14M to 31M, but 31M gains as much from it (Section 4.8). Baselines with the full context window and with matched compute are still missing.

Does it resemble human memory? Only suggestively, and we did not test it directly. Most of a book's memory is on the specific tokens seen (+0.223 for rare tokens of A1, against +0.056 for other rare tokens), which resembles episodic memory, while the gain that survives shuffling transfers to unseen books, which resembles semantic memory. Eviction is fast at first and slow later, closer to a power law than to an exponential.

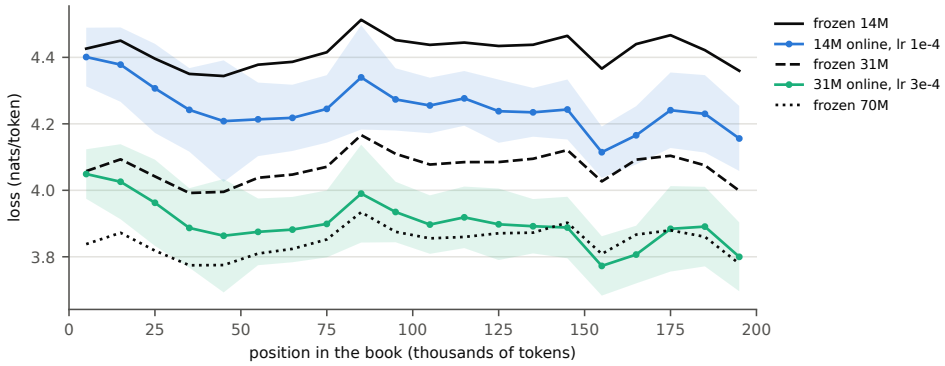


Figure 6: Loss by position in 200k-token books (10k-token bins, mean of 4 books). Bands: ±1 SE over books, which mostly reflect differences in book difficulty; the paired differences in Table 7 are much tighter.

6 Limitations

- **One seed.** Error bars are across books, not across models or runs.
- **Three sizes of one family.** Pythia-14M, 31M and 70M only.
- **One kind of data.** PG-19 books only; WikiText is the only out-of-domain text, and forgetting is measured on 76 WikiText chunks and 6 books.
- **No full-context baseline.** Chunks of 256 tokens are scored without earlier context; a model with its full context window would see more of the recent past through attention.
- **Learning rates.** Rates are picked on mean validation loss, not on locality, and different subsets and sizes use different rates, so comparisons across them mix in the rate.
- **Wall time is not compute.** We did not measure compute; dynamic-evaluation runs update after every chunk, frozen runs do not.
- **Overlapping sets.** All 5 long books and 2 of the 4 very long books are also in the main set.

7 Future work

- A separate fast cache part on top of a frozen or slowly learning model, with a fallback to the frozen model when the cache does not help.
- Capacity: many books interleaved at once, and the size of the adapted part against the eviction half-life.
- Why Adam removes the cache: ablate its momentum and its per-weight normalization.
- Baselines with the full context window and with matched compute.
- More seeds (PolyPythias [23]) and larger models.
- Direct memory tests: savings on re-reading, interference between similar books, spacing, and planted facts.

8 Conclusion

A small language model that takes one update step on each chunk it reads gains a memory of its current context beyond its attention window. The memory behaves like a cache in three ways: it depends on the order of the input (20% of the gain less than 1k tokens back survives shuffling), it is tied to the current book, and other books evict it, halving it after about 6.1k tokens. It differs from a cache in where it is stored: it shares weights with general knowledge, and filling it costs knowledge outside the domain, about 2.0 nats of WikiText loss per

nat gained. Which weights are updated and which update rule is used change how much cache a model gets, but within 14M and 31M not the trade-off between cache and forgetting. Between 14M and 31M, size does not change this picture, and on long books dynamic evaluation closes 49% of the gap to the larger model without replacing it.

References

[1] Jimmy Ba, Geoffrey E. Hinton, Volodymyr Mnih, Joel Z. Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Advances in Neural Information Processing Systems 29 (NIPS)*, 2016.

[2] Rachit Bansal, Aston Zhang, Rishabh Tiwari, Lovish Madaan, Sai Surya Duvvuri, Devvrit Khatri, David Brandfonbrener, David Alvarez-Melis, Prajjwal Bhargava, Mihir Sanjay Kale, and Samy Jelassi. Let's (not) just put things in context: Test-time training for long-context LLMs. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2512.13898.

[3] Stella Biderman, Kieran Bicheno, and Leo Gao. Datasheet for the Pile. arXiv:2201.07311, 2022.

[4] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 2397–2430, 2023.

[5] Peter J. Denning. The working set model for program behavior. *Communications of the ACM*, 11(5):323–333, 1968.

[6] Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Di He, Wenhao Huang, and Tianle Cai. In-place test-time training. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2604.06169.

[7] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027, 2020.

[8] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 5484–5495, 2021.

[9] Edouard Grave, Armand Joulin, and Nicolas Usunier. Improving neural language models with a continuous cache. In *International Conference on Learning Representations (ICLR)*, 2017.

[10] Geoffrey E. Hinton and David C. Plaut. Using fast weights to deblur old memories. In *Proceedings of the Ninth Annual Conference of the Cognitive Science Society*, pages 177–186, Hillsdale, NJ, 1987. Erlbaum.

[11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[12] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *PMLR*, pages 2766–2775, 2018.

[13] Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of transformer language models. arXiv:1904.08378, 2019.

[14] Roland Kuhn and Renato De Mori. A cache-based natural language model for speech recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 12(6):570–583, 1990.

[15] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.

[16] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[17] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020.

[18] Amal Rannen-Triki, Jorg Bornschein, Razvan Pascanu, Marcus Hutter, András György, Alexandre Galashov, Yee Whye Teh, and Michalis K. Titsias. Revisiting dynamic evaluation: Online adaptation for large language models. arXiv:2403.01518, 2024.

[19] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *PMLR*, pages 9355–9366, 2021.

[20] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139, 1992.

[21] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with

self-supervision for generalization under distribution shifts. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 9229–9248, 2020.

[22] Arnuv Tandon, Karan Dalal, Xinhao Li, Daniel Koceja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-end test-time training for long context. arXiv:2512.23675, 2025.

[23] Oskar van der Wal, Pietro Lesci, Max Müller-Eberstein, Naomi Saphra, Hailey Schoelkopf, Willem Zuidema, and Stella Biderman. PolyPythias: Stability and outliers across fifty language model pre-training runs. In *International Conference on Learning Representations (ICLR)*, 2025.